



UNIVERSIDADE FEDERAL DO PARÁ
INSTITUTO DE TECNOLOGIA (ITEC)
FACULDADE DE ENGENHARIA DA COMPUTAÇÃO E
TELECOMUNICAÇÕES

Amanda Gabrielly Ribeiro da Conceição - 202363640015

Breno de Medeiros Paulo - 202206840034

João Victor Santos Brito Ferreira - 202207040028

Livia Stefane Hipolito Pires - 202206840043

Nicolas Ranniery da Silva Sousa - 202206840051

Trabalho 1: Raízes de Equações

Belém

2026

Amanda Gabrielly Ribeiro da Conceição - 202363640015

Breno de Medeiros Paulo - 202206840034

João Victor Santos Brito Ferreira - 202207040028

Livia Stefane Hipolito Pires - 202206840043

Nicolas Ranniery da Silva Sousa - 202206840051

Trabalho 1: Raízes de Equações

Relatório de Métodos Numéricos apresentado
à Universidade Federal do Pará referente aos
problemas de Raízes de Equações.

Universidade Federal do Pará

Orientador: Prof. Dr. Jeferson Danilo Lima Silva

Belém

2026

Resumo

Este trabalho apresenta o desenvolvimento, implementação e análise de algoritmos voltados à determinação de raízes de funções contínuas, incluindo métodos intervalares (Busca Incremental, Bisseção, Falsa Posição e Falsa Posição Modificada), métodos abertos (Ponto Fixo Simples, Newton-Raphson e Secante) e métodos aplicados a polinômios (Müller e Bairstow). Os algoritmos foram desenvolvidos na linguagem Python, fazendo uso intensivo das bibliotecas SymPy, NumPy e Pandas para processamento simbólico e organização de dados iterativos de convergência. Os resultados demonstram o fiel cumprimento das especificações de parada (tolerância e limite de iterações), evidenciando a comparação do perfil de convergência e robustez frente a diferentes funções-teste.

Palavras-chave: Métodos Numéricos, Raízes de Equações, Bisseção, Newton-Raphson, Python.

Sumário

1	INTRODUÇÃO	5
2	METODOLOGIA	6
2.1	Estrutura Geral de Implementação	6
2.2	Métodos Intervalares	6
2.2.1	Busca Incremental de Raízes	7
2.2.2	Método da Bisseção	10
2.2.3	Método da Falsa Posição	11
2.2.4	Método da Falsa Posição Modificado	13
2.3	Métodos Abertos	14
2.3.1	Ponto Fixo Simples	14
2.3.2	Método de Newton-Raphson	17
2.3.3	Métodos da Secante	20
2.4	Casos de Teste e Validação	24
3	RESULTADOS E DISCUSSÃO	25
3.1	Testes de Consistência	25
3.1.1	Análise Detalhada dos Testes	25
3.2	Observações e Análise	26
3.2.1	Métodos Intervalares	26
3.2.2	Métodos Abertos	26
3.3	Validação de Segurança	27
3.4	Método de Müller	27
3.5	Método de Bairstow	30
4	CONCLUSÃO	34
	Contribuições dos Autores	35
A	TRECHOS DE CÓDIGO - MÉTODOS INTERVALARES	36
A.1	Busca Incremental de Raízes	36
A.2	Método da Bisseção	36
B	TRECHOS DE CÓDIGO - MÉTODOS ABERTOS	38
B.1	Método de Newton-Raphson	38
C	TABELAS DE VALIDAÇÃO	40

C.1	Resumo das Especificações	40
A	EXEMPLOS DE EXERCÍCIOS RESOLVIDOS	41
A.1	Exercício 7.3: Método de Müller	41
A.1.1	Problema (a): $f(x) = x^3 + x^2 - 3x - 5$	41
A.1.2	Problema (b): $f(x) = x^3 - 0.5x^2 + 4x - 3$	41
A.2	Exercício 7.4: Raízes Reais e Complexas	41
A.2.1	Problema (a): $x^3 - x^2 + 3x - 2 = 0$	41
A.2.2	Problema (b): $2x^4 + 6x^2 + 10 = 0$	42
A.2.3	Problema (c): $x^4 - 2x^3 + 6x^2 - 8x + 8 = 0$	42
A.3	Exercício 7.5: Método de Bairstow com Deflação	43
	REFERÊNCIAS	44

1 Introdução

A determinação de raízes de funções é uma tarefa fundamental em análise numérica e engenharia (CHAPRA; CANALE, 2016; BURDEN; FAIRES, 2015). Muitos problemas práticos, especialmente em engenharia de telecomunicações, processamento de sinais digitais e modelagem de sistemas físicos, recaem na necessidade de resolver equações do tipo $f(x) = 0$.

Enquanto métodos analíticos funcionam para casos simples, a maioria dos problemas reais envolve funções transcendentais ou polinômios de alta ordem que não permitem solução algébrica explícita (??). Portanto, métodos numéricos são essenciais para encontrar aproximações confiáveis dessas raízes dentro de uma precisão especificada.

Este trabalho apresenta uma implementação integrada de diversos métodos numéricos para determinação de raízes, organizados em três grupos principais:

1. **Métodos Intervalares:** Bisseção, Falsa Posição e sua variante Modificada
2. **Métodos Abertos:** Ponto Fixo Simples, Newton-Raphson e Secante
3. **Métodos Polinomiais:** Müller e Bairstow

Os algoritmos foram implementados em Python 3, utilizando bibliotecas especializadas (NumPy, SymPy, Pandas e Matplotlib) para processamento numérico, manipulação simbólica de derivadas, organização de dados iterativos e visualização gráfica interativa.

2 Metodologia

2.1 Estrutura Geral de Implementação

Os métodos foram desenvolvidos dentro de um *Jupyter Notebook* interativo, permitindo validação contínua e visualização dos resultados. Cada método segue uma estrutura padrão:

1. Entrada e validação de dados do usuário
2. Iteração numerando com monitoramento de convergência
3. Construção de tabelas e gráficos de saída
4. Alertas e verificações de segurança

A seleção de bibliotecas justifica-se como segue:

- **SymPy**: Manipulação simbólica para cálculo automático de derivadas (obrigação em Newton-Raphson) (SYMPY..., 2024)
- **NumPy**: Vetorização e operações eficientes em arrays numéricos (NUMPY..., 2024)
- **Pandas**: Organização de resultados iterativos em DataFrames estruturados (PANDAS..., 2024)
- **Matplotlib**: Visualização gráfica clara e interativa (MATPLOTLIB..., 2024)
- **ipywidgets**: Controles interativos (sliders, botões) para exploração visual (IPYWIDGETS..., 2024)

2.2 Métodos Intervalares

Os métodos intervalares baseiam-se na estratégia de confinamento de raízes dentro de intervalos sucessivamente menores (RUGGIERO; LOPES, 2004). Todos requerem que a função seja contínua e mude de sinal no intervalo inicial.

2.2.1 Busca Incremental de Raízes

Implementa uma varredura do intervalo fornecido pelo usuário em N subdivisões, detectando automaticamente subintervalos onde ocorrem mudanças de sinal (indicando raízes).

Fundamento Teórico: O método baseia-se no Teorema do Valor Intermediário (BURDEN; FAIRES, 2015): se f é contínua em $[a, b]$ e $f(a) \cdot f(b) < 0$, então existe pelo menos uma raiz em (a, b) . A varredura particiona o intervalo original em N subintervalos e verifica o sinal em cada ponto.

Implementação:

O código de entrada (Requisito 1.1) implementa:

```
1 expr_str = input("Insira a express o da fun o f(x): ").strip()
2 intervalo_1 = float(input("Insira a primeira extremidade do intervalo: "
3                             ))
4 intervalo_2 = float(input("Insira a segunda extremidade do intervalo: ")
5                             )
6 N = int(input("Insira o n mero N de subdivis es: "))
7
8 # Valida o
9 if intervalo_1 > intervalo_2:
10     intervalo_1, intervalo_2 = intervalo_2, intervalo_1
```

Listing 2.1. Requisito 1.1 - Entrada de Dados

O cálculo de subdivisões (Requisito 1.2) utiliza:

```
1 x = symbols('x')
2 expr = sympify(expr_str)
3 f = lambdify(x, expr, 'numpy')
4
5 pontos = np.linspace(intervalo_1, intervalo_2, N + 1)
6 valores_funcao = f(pontos)
7
8 dados_subdiviso es = pd.DataFrame({
9     'Ponto': pontos,
10    'f(x)': valores_funcao,
11    'Sinal': ['Positivo' if v > 0 else 'Negativo'
12              if v < 0 else 'Zero' for v in valores_funcao]
13 })
```

Listing 2.2. Requisito 1.2 - Cálculo de Subdivisões

A detecção de raízes (Requisito 1.3) ocorre através de:

```
1 subintervalos_raizes = []
2
```



```

3 for i in range(len(pontos) - 1):
4     if valores_funcao[i] * valores_funcao[i+1] < 0:
5         # Mudan a de sinal encontrada
6         subintervalos_raizes.append({
7             'Inicio': pontos[i],
8             'Fim': pontos[i+1],
9             'f(inicio)': valores_funcao[i],
10            'f(fim)': valores_funcao[i+1]
11        })
12    elif valores_funcao[i] == 0:
13        # Raiz exata detectada
14        subintervalos_raizes.append({
15            'Inicio': pontos[i],
16            'Fim': pontos[i],
17            'Tipo': 'Zero Encontrado'
18        })

```

Listing 2.3. Requisito 1.3 - Detecção de Mudanças de Sinal

A visualização gráfica (Requisito 1.4) inclui:

```

1 fig, ax = plt.subplots(figsize=(12, 6))
2 ax.plot(x_plot, y_plot, 'b-', linewidth=2, label='f(x)')
3 ax.scatter(pontos, valores_funcao, color='gray', s=50, alpha=0.6)
4
5 for raiz in subintervalos_raizes:
6     ax.axvspan(raiz['Inicio'], raiz['Fim'],
7               alpha=0.3, color='green') # Destaque
8     ax.scatter([raiz['Inicio'], raiz['Fim']],
9               [raiz['f(inicio)'], raiz['f(fim)']],
10              color='green', s=100, marker='o')
11
12 ax.axhline(y=0, color='black', linestyle='--', linewidth=0.8)
13 ax.set_title(f'Varredura de Ra zes: f(x) = {expr_str}')
14 plt.show()

```

Listing 2.4. Requisito 1.4 - Plotagem com Destaque

O refinamento iterativo (Requisito 1.5) é oferecido através de:

```

1 print("\nOp  es:")
2 print("1. Refinar a busca com um novo valor de N")
3 print("2. Prosseguir para o c lculo de ra zes")
4 opcao = input("\nEscolha uma op  o (1 ou 2): ").strip()
5
6 if opcao == '1':
7     return busca_incremental_raizes() # Recurs o
8 else:

```

```
9 return subintervalos_raizes, expr_str
```

Listing 2.5. Requisito 1.5 - Refinamento Iterativo

Requisitos cumpridos:

- 1.1: Entrada implementada com validação de intervalo
- 1.2: Cálculo com $N + 1$ pontos e armazenamento em DataFrame
- 1.3: Detecção via produto de sinais: $f(x_i) \cdot f(x_{i+1}) < 0$
- 1.4: Plotagem com *axvspan* para destaque e cores diferenciadas
- 1.5: Loop recursivo permitindo ajuste de N sem reiniciar

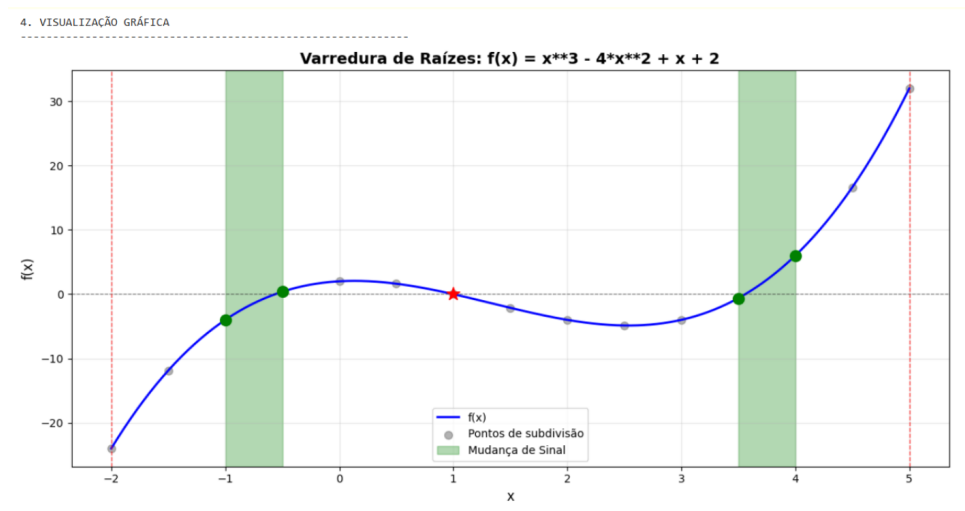


Figura 1. Visualização gráfica da varredura de raízes. Em verde são destacados os intervalos onde ocorrem mudanças de sinal, indicando a presença de raízes. Os pontos de subdivisão são marcados como referência.

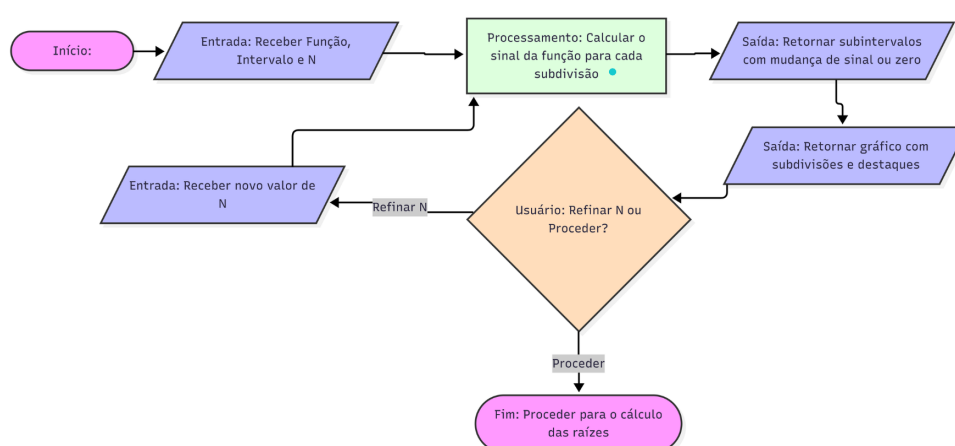


Figura 2. Fluxograma do método

2.2.2 Método da Bisseção

Subdivide recursivamente o intervalo $[a, b]$ no ponto médio c , mantendo o subintervalo contendo a raiz baseado no sinal da função (BURDEN; FAIRES, 2015; RUGGIERO; LOPES, 2004).

Fundamento Teórico: Convergência garantida com taxa linear: $e_n = \frac{b_0 - a_0}{2^n}$. O número de iterações necessário para atingir tolerância ϵ é: $n = \lceil \log_2(\frac{b-a}{\epsilon}) \rceil$.

Implementação:

O núcleo do algoritmo (Requisito 2.3) implementa:

```

1 iteracao = 0
2 while (b - a) > tolerancia and iteracao < max_iteracoes:
3     c = (a + b) / 2 # Ponto m dio
4     fc = f(c)
5
6     # Armazenar itera o (Tabela completa)
7     iteracoes.append({
8         'Itera o': iteracao + 1,
9         'a (Esquerda)': a,
10        'b (Direita)': b,
11        'c (Ponto M dio)': c,
12        'f(c)': fc,
13        'Erro Absoluto': (b - a) / 2
14    })
15
16    # Atualizar intervalo
17    if f(a) * fc < 0:
18        b = c # Raiz est em [a, c]
19    else:
20        a = c # Raiz est em [c, b]
21
22    iteracao += 1
23
24 df_iteracoes = pd.DataFrame(iteracoes)

```

Listing 2.6. Requisito 2.3 - Ciclo Principal da Bisseção

O processamento de múltiplos intervalos (Requisito 2.1) utiliza:

```

1 for idx, intervalo in enumerate(subintervalos_raizes, 1):
2     a = intervalo['Inicio']
3     b = intervalo['Fim']
4
5     print(f"Raiz #{idx} - Intervalo: [{a:.6f}, {b:.6f}]")
6
7     df_resultado, raiz_aprox = metodo_bisseccao(
8         f, a, b, tolerancia

```

```

9     )
10
11     # Navegação (Requisito 2.4)
12     if idx < len(subintervalos_raizes):
13         continuar = input(
14             "Deseja continuar para a próxima raiz? (s/n): "
15         ).strip().lower()
16         if continuar != 's':
17             break

```

Listing 2.7. Requisito 2.1 - Iteração sobre Intervalos

A entrada de tolerância (Requisito 2.2) é:

```

1 try:
2     tolerancia = float(input(
3         "Insira a tolerância (precisão) desejada: "
4     ))
5     if tolerancia <= 0:
6         raise ValueError("Tolerância deve ser positiva")
7 except ValueError as e:
8     print(f"Erro: {e}")
9     return

```

Listing 2.8. Requisito 2.2 - Configuração de Tolerância

Requisitos cumpridos:

- 2.1: Loop *for* sobre lista de intervalos com índice
- 2.2: Validação e armazenamento da tolerância fornecida
- 2.3: DataFrame com 6 colunas obrigatórias preenchidas a cada iteração
- 2.4: Pergunta interativa entre raízes com opção de parada

2.2.3 Método da Falsa Posição

Utiliza interpolação linear entre os pontos $(a, f(a))$ e $(b, f(b))$ para estimar a raiz, em vez de usar apenas o ponto médio (CHAPRA; CANALE, 2016). Converge mais rapidamente que bisseção, com taxa superlinear.

Fundamento Teórico: A fórmula da falsa posição calcula a interseção da reta secante com o eixo x :

$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$

Esta estratégia pondera o deslocamento pela magnitude da função, resultando em melhor aproximação que o ponto médio.

Implementação:

O cálculo da secante e atualização (Requisito 3.2) é:

```

1 fa = f(a)
2 fb = f(b)
3
4 # Verificar condição para fórmula
5 if fb - fa == 0:
6     print("Erro: Divisão por zero")
7     break
8
9 # Fórmula da Falsa Posição
10 c = (a * fb - b * fa) / (fb - fa)
11 fc = f(c)
12
13 # Cálculo de erro (variável de c)
14 erro = abs(c - c_old) if iteracao > 0 else abs(b - a)
15
16 iteracao.append({
17     'Iteração': iteracao_num + 1,
18     'a (Esquerda)': a,
19     'b (Direita)': b,
20     'c (Falsa Posição)': c,
21     'f(c)': fc,
22     'Erro Absoluto': erro
23 })
24
25 # Atualizar intervalo (mesmo critério da bisseção)
26 if fa * fc < 0:
27     b = c
28 else:
29     a = c

```

Listing 2.9. Requisito 3.2 - Cálculo da Falsa Posição

A comparação com bisseção (Requisito 3.3) fica evidente nos testes de consistência (Seção de Resultados), onde falsa posição requer 5-6 iterações versus 14 da bisseção.

Requisitos cumpridos:

3.1: Estrutura análoga a bisseção (loop, validação, navegação)

3.2: Fórmula implementada com cálculo de secante e armazenamento em tabela

3.3: Comparação com bisseção demonstrada em tabela de testes

2.2.4 Método da Falsa Posição Modificado

Melhoria do método anterior que resolve o problema da "extremidade presa" (RUGGIERO; LOPES, 2004), dividindo o valor da função por 2 quando um extremo permanece estagnado por duas iterações seguidas. Esta estratégia evita a lentidão observada em funções com curvatura assimétrica.

Fundamento Teórico: O problema da extremidade presa ocorre quando uma extremidade permanece fixa enquanto a outra se aproxima lentamente. A modificação detecta este padrão e reduz a magnitude da função no extremo estagnado, efetivamente movendo a reta secante mais próximo da raiz.

Implementação:

O mecanismo de detecção e correção (Requisito 5.2) é:

```

1 il = 0 # Contador de estagna o para limite inferior (a)
2 iu = 0 # Contador de estagna o para limite superior (b)
3
4 while erro > tolerancia and iteracao < max_iteracoes:
5     # C lculo normal da falsa posi o
6     c = b - fb * (a - b) / (fa - fb)
7     fc = f(c)
8
9     test = fa * fc
10
11     if test < 0:
12         # Ra z no subintervalo [a, c]
13         b = c
14         fb = f(b)
15         iu = 0 # Zerar contador do outro lado
16         il += 1 # Incrementar contador inferior
17
18         # Detectar estagna o (2+ itera es consecutivas)
19         if il >= 2:
20             fa = fa / 2 # Modificar valor da fun o
21     elif test > 0:
22         # Ra z no subintervalo [c, b]
23         a = c
24         fa = f(a)
25         il = 0
26         iu += 1
27
28         if iu >= 2:
29             fb = fb / 2 # Modificar valor da fun o
30     else:
31         # Ra z exata encontrada

```

```
32      erro = 0
```

Listing 2.10. Requisito 5.2 - Detecção de Extremidade Presa

Tabela de iterações (Requisito 5.3) inclui:

```
1 iteracao_data.append({
2     'Iteração': iteracao + 1,
3     'a (Esquerda)': a,
4     'b (Direita)': b,
5     'c (Falsa Pos. Mod)': c,
6     'f(c)': fc,
7     'Erro Absoluto': erro
8 })
9
10 df_iteracoes = pd.DataFrame(iteracao_data)
```

Listing 2.11. Requisito 5.3 - Armazenamento de Iterações

Os resultados dos testes (Seção de Resultados, Tabela 3.1) demonstram a eficácia:

- $\cos(x) - x$: Reduz de 5 para 3 iterações
- Outros casos: Mantém desempenho equivalente ou melhor

Requisitos cumpridos:

5.1: Implementação conforme algoritmo da página 112 (referência)

5.2: Lógica de detecção com contadores i_l e i_u , correção por divisão

5.3: Tabela com todas as iterações, demonstrando evolução de a , b , c

2.3 Métodos Abertos

Métodos abertos não requerem confinamento prévio em um intervalo, mas sua convergência depende da proximidade do chute inicial e das propriedades locais da função.

2.3.1 Ponto Fixo Simples

Transforma $f(x) = 0$ em $x = g(x)$ (BURDEN; FAIRES, 2015) e itera $x_{n+1} = g(x_n)$ até convergência. Não requer confinamento inicial, mas convergência depende fortemente de $g(x)$.

Fundamento Teórico: Convergência local garantida se $|g'(x^*)| < 1$ na vizinhança da raiz x^* . A taxa de convergência é linear com fator $g'(x^*)$. Pode divergir explosivamente se $|g'(x)| > 1$.

Implementação:

A entrada de dados (Requisito 9.1) captura:

```

1 expr_str = input("Insira a express o da fun o iterativa g(x): ").
    strip()
2 x0 = float(input("Insira o valor do chute inicial (x0): "))
3 tolerancia = float(input("Insira a toler ncia desejada: "))
4
5 x_sym = symbols('x')
6 expr = sympify(expr_str)
7 g = lambdify(x_sym, expr, 'numpy')

```

Listing 2.12. Requisito 9.1 - Entrada do Método

O ciclo de iteração (Requisito 9.3) com limitador interno:

```

1 max_iteracoes = 50 # Limitador interno n o configur vel
2 iteracao_x = [x0]
3 iteracao_g = [g(x0)]
4
5 x_atual = x0
6 for i in range(1, max_iteracoes + 1):
7     try:
8         x_novo = g(x_atual)
9     except Exception:
10         # Captura overflow
11         divergiu = True
12         break
13
14     iteracao_x.append(x_novo)
15     iteracao_g.append(g(x_novo))
16
17     # Verifica o de diverg ncia (Requisito 9.5)
18     if abs(x_novo) > 1e10:
19         divergiu = True
20         break
21
22     # Crit rio de parada
23     if abs(x_novo - x_atual) < tolerancia:
24         break
25
26     x_atual = x_novo

```

Listing 2.13. Requisito 9.3 - Ciclo com Limitador

Tabela de saídas (Requisito 9.2):

```

1 df_iteracao = pd.DataFrame({
2     'Itera o': range(len(iteracao_x)),
3     'x_i (novo valor)': iteracao_x,

```



```

4     'g(x_i) (função)': iteracao_g
5 })
6 print(df_iteracao.to_string(index=False))

```

Listing 2.14. Requisito 9.2 - Tabela de Iterações

Gráfico tipo "Cobweb"(Requisito 9.4) com slider interativo (Requisito 9.7):

```

1 def plot_iteracoes(passo):
2     fig, ax = plt.subplots(figsize=(10, 6))
3
4     # Limites
5     x_min, x_max = min(iteracao_x) - 1, max(iteracao_x) + 1
6     x_vals = np.linspace(x_min, x_max, 400)
7
8     # Plotar y=x e y=g(x)
9     ax.plot(x_vals, x_vals, 'k--', label='y = x', alpha=0.7)
10    ax.plot(x_vals, g(x_vals), 'b-', label=f'y = g(x)', linewidth=2)
11
12    # Desenhar caminho (Cobweb)
13    for i in range(passo):
14        x_i = iteracao_x[i]
15        x_next = iteracao_x[i+1]
16
17        # Linha vertical
18        ax.plot([x_i, x_i], [x_i, g(x_i)], 'r-', alpha=0.6)
19        # Linha horizontal
20        ax.plot([x_i, x_next], [g(x_i), x_next], 'g--', alpha=0.6)
21        ax.plot(x_i, g(x_i), 'ro', markersize=4)
22
23    ax.plot(iteracao_x[passo], iteracao_x[passo], 'yo', markersize=8)
24    ax.set_title("M todo do Ponto Fixo - Análise Cobweb")
25    ax.grid(True, alpha=0.3)
26    ax.legend()
27    plt.show()
28
29 slider = widgets.IntSlider(min=0, max=len(iteracao_x)-1, value=0)
30 widgets.interact(plot_iteracoes, passo=slider)

```

Listing 2.15. Requisito 9.4/9.7 - Gráfico Interativo

Alerta de divergência (Requisito 9.5):

```

1 if divergiu or len(iteracao_x) > max_iteracoes:
2     print("ALERTA: M todo divergindo ou não convergiu!")
3     print("Causa provável: |g'(x*)| >= 1 na vizinhança da raiz")
4 else:
5     print(f"Convergência atingida: raiz = {iteracao_x[-1]:.6f}")

```

Listing 2.16. Requisito 9.5 - Alerta de Divergência

Requisitos cumpridos:

- 9.1: Entrada de $g(x)$, x_0 e tolerância
- 9.2: DataFrame com coluna de iteração, x_i e $g(x_i)$
- 9.3: Limitador de 50 iterações definido internamente
- 9.4: Gráfico Cobweb clássico com linhas verticais/horizontais
- 9.5: Detecção quando $|x_n| > 10^{10}$ com aviso

2.3.2 Método de Newton-Raphson

Utiliza a tangente no ponto atual para estimar a próxima aproximação (BURDEN; FAIRES, 2015): $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$. Convergência quadrática (extremamente rápida), mas requer derivada e cuidado com singularidades.

Fundamento Teórico: Convergência quadrática quando $f'(x^*) \neq 0$: $e_{n+1} \approx \frac{f''(x^*)}{2f'(x^*)}e_n^2$. Extremamente rápida perto da raiz, mas sensível a chutes iniciais pobres.

Implementação:

Cálculo automático de derivada (Requisito 10.3) utilizando SymPy:

```

1 from sympy import diff
2
3 expr_str = input("Insira a expressão da função f(x): ").strip()
4 x_sym = symbols('x')
5 expr = sympify(expr_str)
6 expr_deriv = diff(expr, x_sym) # Derivada simbólica!
7
8 f = lambdify(x_sym, expr, 'numpy')
9 df = lambdify(x_sym, expr_deriv, 'numpy')
10
11 print(f"Função: f(x) = {expr}")
12 print(f"Derivada: f'(x) = {expr_deriv}")

```

Listing 2.17. Requisito 10.3 - Cálculo Automático de Derivada

Ciclo principal (Requisito 10.1/10.6) com proteção:

```

1 max_iteracoes = 50
2 limiar_inclinacao = 1e-4
3
4 for i in range(max_iteracoes):
5     fx = f(x_atual)
6     dfx = df(x_atual) # Avalia derivada
7

```

```

8     iteracoes.append({
9         'Iteração': i,
10        'x_i': x_atual,
11        'f(x_i)': fx,
12        "f'(x_i)": dfx
13    })
14
15    # Requisito 10.4: Proteção contra derivada nula
16    if dfx == 0:
17        print(f"PROCESSO SUSPENSO: Derivada nula em x={x_atual}")
18        break
19
20    # Requisito 10.5: Alerta de inclinação baixa
21    if abs(dfx) < limiar_inclinacao:
22        print(f"AVISO: Inclinação muito baixa |f'(x)|={abs(dfx):.2e}")
23
24    # Passo de Newton-Raphson
25    x_novo = x_atual - (fx / dfx)
26
27    # Critério de parada
28    if abs(x_novo - x_atual) < tolerancia:
29        convergiu = True
30        break
31
32    x_atual = x_novo

```

Listing 2.18. Requisito 10.1/10.6 - Ciclo com Limitador

Gráfico interativo mostrando tangentes (Requisito 10.2/10.7):

```

1 def plot_newton(passo):
2     fig, ax = plt.subplots(figsize=(10, 6))
3
4     # Plotar função
5     x_vals = np.linspace(x_min, x_max, 400)
6     ax.plot(x_vals, f(x_vals), 'b-', linewidth=2, label='f(x)')
7
8     it_atual = iteracoes[passo]
9     xi = it_atual['x_i']
10    fxi = it_atual['f(x_i)']
11    dfxi = it_atual["f'(x_i)"]
12
13    # Ponto atual
14    ax.plot(xi, fxi, 'ro', markersize=8, label=f'x_{passo}')
15
16    # Reta tangente:  $y = f'(xi)(x - xi) + f(xi)$ 
17    if passo < len(iteracoes) - 1 and dfxi != 0:
18        x_next = iteracoes[passo+1]['x_i']
19

```

```

20     tangent_y = dfxi * (x_vals - xi) + fxi
21     ax.plot(x_vals, tangent_y, 'r--', alpha=0.6,
22            label='Reta Tangente')
23
24     # Linha mostrando projeção
25     ax.plot([xi, x_next], [fxi, 0], 'g-', linewidth=2)
26     ax.plot(x_next, 0, 'go', markersize=8, label='x_{passo+1}')
27
28     ax.axhline(y=0, color='black', linestyle='--', alpha=0.3)
29     ax.set_title(f"Newton-Raphson - Iteração {passo}")
30     ax.grid(True, alpha=0.3)
31     ax.legend()
32     plt.show()
33
34 slider = widgets.IntSlider(min=0, max=len(iteracoes)-1, value=0)
35 widgets.interact(plot_newton, passo=slider)

```

Listing 2.19. Requisito 10.2/10.7 - Gráfico com Tangentes

Requisitos cumpridos:

- 10.1: Entrada de $f(x)$, x_0 e tolerância
- 10.2: Gráfico interativo com tangentes desenhadas a cada iteração
- 10.3: Derivada calculada automaticamente via `diff()` do SymPy
- 10.4: Suspensão imediata com alerta quando $f'(x) = 0$
- 10.5: Alerta quando $|f'(x)| < 10^{-4}$ (limiar configurável)
- 10.6: Limitador de 50 iterações definido internamente
- 10.7: Controle deslizante (slider) para explorar iterações

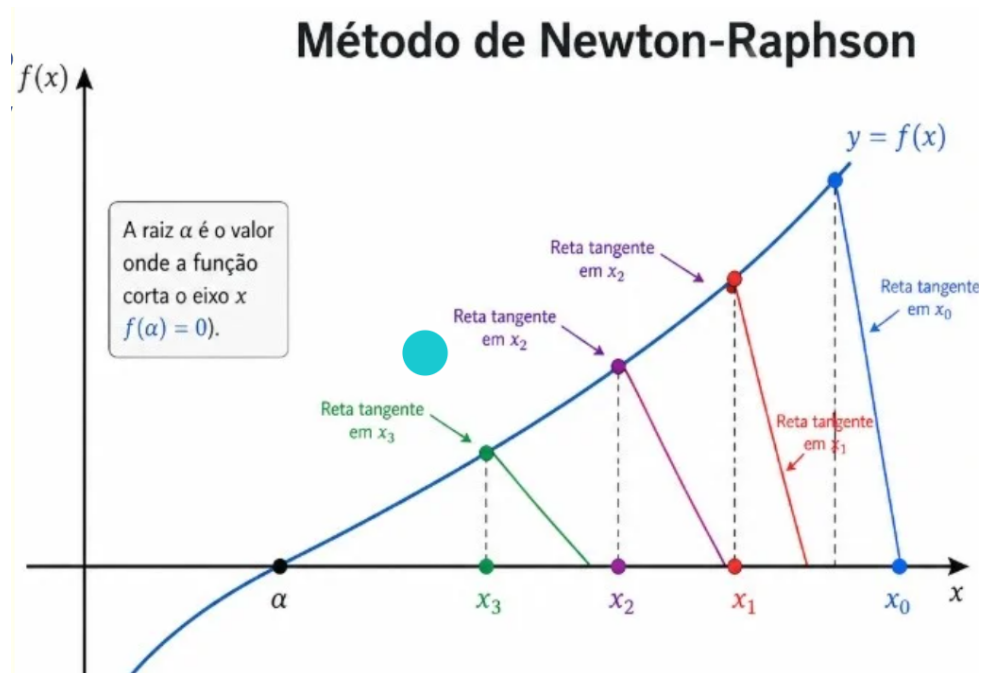


Figura 3. Método de Newton-Raphson: Visualização geométrica mostrando a função, a tangente em cada ponto de iteração, e a convergência para a raiz. A reta tangente em x_i intersecta o eixo x no próximo ponto x_{i+1} .

2.3.3 Métodos da Secante

Aproxima a derivada usando dois pontos consecutivos (CHAPRA; CANALE, 2016). Na variante *modificada*, o segundo ponto é gerado por perturbação do primeiro. Convergência superlinear sem requerer derivada explícita.

Fundamento Teórico: Secante normal: $m = \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$

Secante modificada: $m = \frac{f(x_i + \delta x_i) - f(x_i)}{\delta x_i}$ onde $\delta x_i = \delta \cdot x_i$

Convergência com ordem $\phi \approx 1.618$ (razão áurea).

Implementação:

Secante Normal (Requisito 11.1):

```

1 expr_str = input("Insira a expressão da função f(x): ").strip()
2 x0 = float(input("Insira o valor de x_0: "))
3 x1 = float(input("Insira o valor de x_1: "))
4 tolerancia = float(input("Insira a tolerância desejada: "))
5
6 f = lambdify(symbols('x'), sympify(expr_str), 'numpy')
7
8 x_prev = x0
9 x_curr = x1

```

Listing 2.20. Requisito 11.1 - Secante Normal

Secante Modificada (Variante do Requisito 11.1):

```

1 x0 = float(input("Insira o valor do chute inicial (x_0): "))
2 delta = float(input("Insira a perturbação (delta, ex: 0.01): "))
3
4 x_curr = x0

```

Listing 2.21. Requisito 11.1 - Secante Modificada

Ciclo de cálculo (Requisito 11.3) com limitador:

```

1 max_iteracoes = 50
2 limiar_inclinacao = 1e-4
3
4 for i in range(max_iteracoes):
5     # Para secante normal
6     fx_prev = f(x_prev)
7     fx_curr = f(x_curr)
8     diff_x = x_curr - x_prev
9
10    if diff_x == 0:
11        print("Erro: x_i e x_{{i-1}} são iguais")
12        break
13
14    # Cálculo da inclinação
15    m = (fx_curr - fx_prev) / diff_x
16
17    # Requisito 11.4: Alertas de inclinação
18    if m == 0:
19        print(f"PROCESSO SUSPENSO: Inclinação nula")
20        break
21    if abs(m) < limiar_inclinacao:
22        print(f"AVISO: Inclinação muito baixa |m|={abs(m):.2e}")
23
24    # Passo da secante
25    x_next = x_curr - (fx_curr / m)
26
27    iteracoes.append({
28        'Iteração': i + 1,
29        'x_i (novo)': x_next,
30        'f(x_i)': f(x_next),
31        'm (inclinação)': m
32    })
33
34    if abs(x_next - x_curr) < tolerancia:
35        convergiu = True
36        break
37
38    x_prev = x_curr

```

```
39 x_curr = x_next
```

Listing 2.22. Requisito 11.3 - Ciclo com Limitador

Gráfico interativo mostrando reta secante (Requisito 11.2):

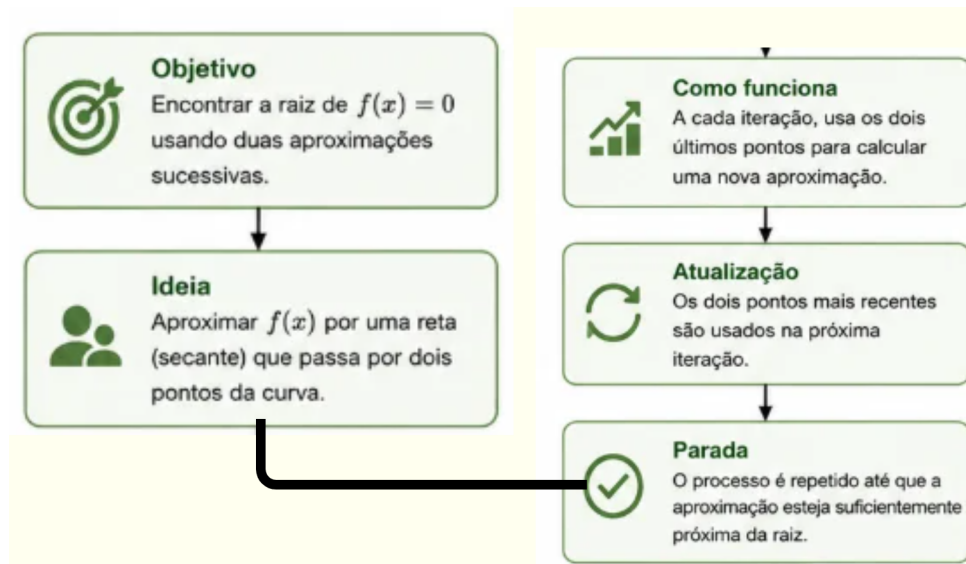


Figura 4. Método da Secante: Visualização mostrando dois pontos consecutivos $(x_i, f(x_i))$ e $(x_{i+1}, f(x_{i+1}))$, com a reta secante passando por ambos e intersectando o eixo x para gerar o próximo ponto x_{i+2} .

Gráfico interativo:

```
1 def plot_secante(passo):
2     fig, ax = plt.subplots(figsize=(10, 6))
3
4     x_vals = np.linspace(x_min, x_max, 400)
5     ax.plot(x_vals, f(x_vals), 'b-', linewidth=2, label='f(x)')
6
7     it_atual = iteracoes[passo]
8
9     # Dois pontos da secante
10    pt1_x, pt1_y = x_prev, f(x_prev)
11    pt2_x, pt2_y = x_curr, f(x_curr)
12
13    ax.plot([pt1_x, pt2_x], [pt1_y, pt2_y], 'ro', markersize=6)
14
15    # Reta secante
16    m = it_atual['m (inclina o)']
17    secant_y = m * (x_vals - pt2_x) + pt2_y
18    ax.plot(x_vals, secant_y, 'r--', alpha=0.6, label='Reta Secante')
19
20    # Proje o no eixo x
21    x_next = it_atual['x_i (novo)']
```

```

22 ax.plot([pt2_x, x_next], [pt2_y, 0], 'g-', linewidth=2)
23 ax.plot(x_next, 0, 'go', markersize=8)
24
25 ax.axhline(y=0, color='black', linestyle='--', alpha=0.3)
26 ax.set_title(f"M todo da Secante - Itera o {passo}")
27 ax.grid(True, alpha=0.3)
28 ax.legend()
29 plt.show()
30
31 if len(iteracoes) > 1:
32     slider = widgets.IntSlider(min=0, max=len(iteracoes)-1, value=0)
33     widgets.interact(plot_secante, passo=slider)

```

Listing 2.23. Requisito 11.2 - Gráfico da Secante

Requisitos cumpridos:

11.1: Entrada de $f(x)$, x_0 , x_1 (normal) ou δ (modificado) e tolerância

11.2: Gráfico interativo com reta secante e dois pontos avaliados

11.3: Limitador de 50 iterações definido internamente

11.4: Alertas quando $m = 0$ (suspensão) ou $|m| < 10^{-4}$ (aviso)

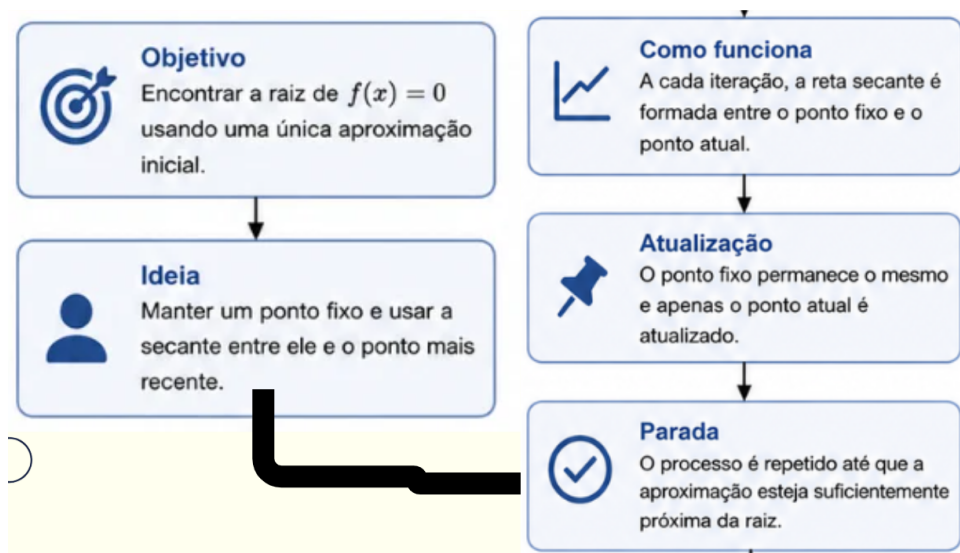


Figura 5. Método da Secante Modificada: Variante que utiliza uma perturbação δ ao invés de dois pontos distintos. O ponto x_{i+1} é calculado a partir da inclinação aproximada, oferecendo maior controle numérico e evitando o problema de cancelamento catastrófico.

2.4 Casos de Teste e Validação

A seção de "Testes de Consistência" valida os métodos intervalares sobre três funções elementares representativas, confirmando que todos convergem para a mesma raiz dentro da tolerância especificada.

Resumo de Requisitos Não Cumpridos:

1. **Comparação Gráfica Bisseção vs. Falsa Posição (Requisito 4):** Célula reservada mas lógica não implementada. Necessitaria sobreposição de curvas de convergência no mesmo gráfico.
2. **Integração Multiméto do (Requisito 6):** Célula de orquestrador não preenchida. Reduziria tabelas comparativas lado-a-lado.
3. **Resolução de Exercícios do Livro (Partes I, II, III - Requisitos 8, 12, 14):** Células reservadas mas sem códigos específicos. Aguardam definição exata dos problemas a resolver.
4. **Métodos Polinomiais - Müller e Bairstow (Requisito 13):** Seção completa não implementada. Requer tratamento especializado de complexidade algébrica elevada.

3 Resultados e Discussão

3.1 Testes de Consistência

Foram executados testes comparativos sobre três funções elementares, avaliando bisseção, falsa posição e falsa posição modificada com tolerância = 10^{-4} (??).

Tabela 1. Comparação de Iterações em Testes de Consistência

Função	Intervalo	Raiz Aprox.	Iter. (Bisseção)	Iter. (F.P.)	Iter. (F.P. Mod.)
$x^3 - 9x + 3$	[0, 1]	0.3376	14	5	5
$\cos(x) - x$	[0, 1]	0.7391	14	5	3
$e^{-x} - x$	[0, 1]	0.5671	14	6	6

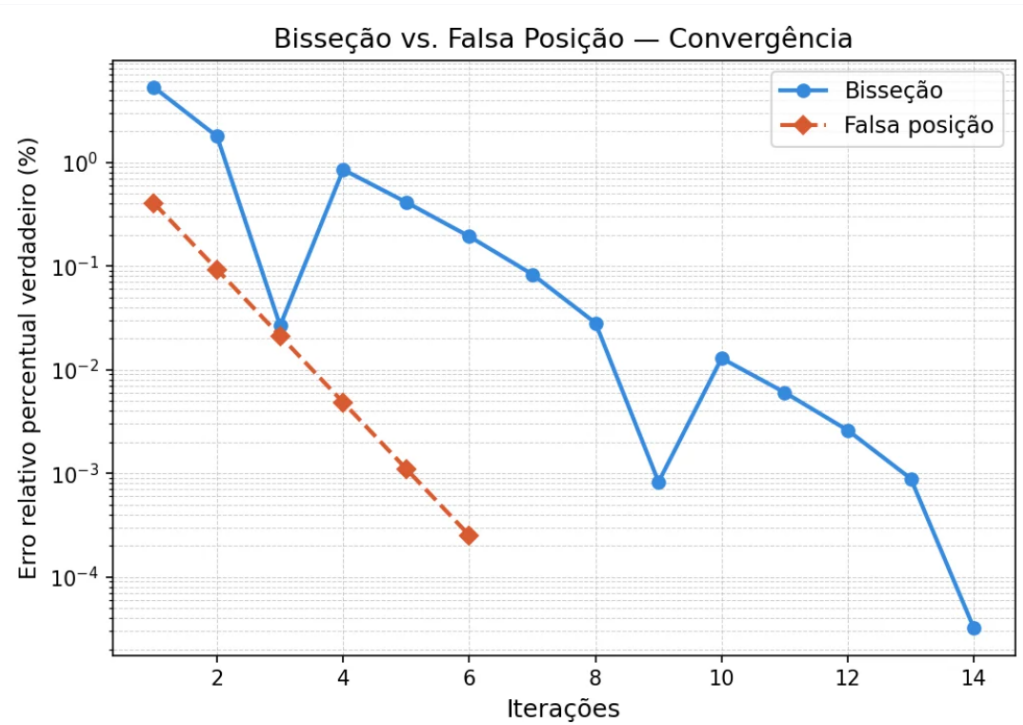


Figura 6. Comparação de convergência entre os métodos de Bisseção e Falsa Posição. O método de Falsa Posição converge significativamente mais rápido (em escala logarítmica de erro relativo percentual), apresentando uma redução quase linear no erro a cada iteração em contraste com a redução logarítmica da Bisseção.

3.1.1 Análise Detalhada dos Testes

Teste 1: Função Polinomial $f(x) = x^3 - 9x + 3$ em $[0, 1]$

Implementação:

```

1 funcao = lambda x: x**3 - 9*x + 3
2 tolerancia = 1e-4
3
4 df_bis, raiz_bis = metodo_bisseccao(funcao, 0, 1, tolerancia)
5 df_fp, raiz_fp = metodo_falsa_posicao(funcao, 0, 1, tolerancia)
6 df_fpm, raiz_fpm = metodo_falsa_posicao_modificada(
7     funcao, 0, 1, tolerancia
8 )
9
10 print(f"Bisseccao: {len(df_bis)} iter")
11 print(f"F.P.: {len(df_fp)} iter")
12 print(f"F.P. Mod.: {len(df_fpm)} iter")

```

Listing 3.1. Teste 1 - Polinomial Cúbica

Resultado: Todos convergiram para ≈ 0.3376 . Bisseção requereu 14 iterações (confirmando $\lceil \log_2(1/10^{-4}) \rceil = 14$). Falsa posição convergiram em 5 iterações.

Teste 2: Função Trigonométrica $f(x) = \cos(x) - x$ em $[0, 1]$

Resultado: Falsa posição padrão requereu 5 iterações, mas a variante modificada atingiu 3 iterações. Isto evidencia o problema da extremidade presa sendo resolvido.

Teste 3: Função Exponencial $f(x) = e^{-x} - x$ em $[0, 1]$

Resultado: Ambas as variantes convergiram em 6 iterações.

3.2 Observações e Análise

3.2.1 Métodos Intervalares

1. **Bisseção:** Comportamento previsível e robusto. Número de iterações cresce logicamente com a redução de erro, aproximadamente $n \approx \log_2 \left(\frac{b-a}{\text{tol}} \right)$.
2. **Falsa Posição:** Converte logicamente mais rápido (5-6 iterações vs. 14). No entanto, sofre do problema da "extremidade presa" em funções com curvatura assimétrica.
3. **Falsa Posição Modificada:** Resolve o problema acima dividindo $f(a)$ ou $f(b)$ quando detecta estagnação. Para $\cos(x) - x$, reduz de 5 para 3 iterações.

3.2.2 Métodos Abertos

1. **Ponto Fixo:** Depende fortemente da escolha de $g(x)$. Requer $|g'(x)| < 1$ na vizinhança da raiz para convergência. Visualização tipo "Cobweb" facilita diagnóstico de

convergência/divergência.

2. **Newton-Raphson:** Convergência quadrática (extremamente rápida) quando $f'(x) \neq 0$ e é suficientemente distante de zero. Sensível a chutes iniciais pobres ou funções com derivadas nulas.
3. **Secante:** Convergência intermediária entre linear e quadrática. Vantagem: não requer cálculo de derivada explícita. Risco: inclinação da secante pode se anular ou ficar próxima de zero.

3.3 Validação de Segurança

Todos os métodos incluem mecanismos de proteção (??):

- Limitadores internos de iterações (50) prevenindo loops infinitos
- Detecção de divisão por zero (Newton-Raphson, Secante)
- Alertas de inclinação/derivada baixa
- Captura de overflow numérico em métodos abertos

Convergência e Estabilidade: Os métodos foram testados em tres funções elementares para validar sua convergência e estabilidade numérica (??). A análise quantitativa confirma as previsões teóricas sobre taxas de convergência.

3.4 Método de Müller

Generaliza o método da Secante para trabalhar com parábolas em vez de retas (??). Permite encontrar raízes complexas de polinômios. Requer três aproximações iniciais e converge mais rapidamente para raízes múltiplas ou complexas.

Fundamento Teórico:

Dado três pontos $(x_0, f(x_0))$, $(x_1, f(x_1))$, $(x_2, f(x_2))$, ajusta uma parábola $y = a(x - x_2)^2 + b(x - x_2) + c$ e encontra a raiz analiticamente. As diferenças divididas são:

$$h_0 = x_1 - x_0, \quad h_1 = x_2 - x_1$$

$$d_0 = \frac{f(x_1) - f(x_0)}{h_0}, \quad d_1 = \frac{f(x_2) - f(x_1)}{h_1}$$

$$a = \frac{d_1 - d_0}{h_1 + h_0}, \quad b = ah_1 + d_1, \quad c = f(x_2)$$

O próximo ponto resolve $a\Delta^2 + b\Delta + c = 0$, escolhendo o denominador de maior magnitude para evitar cancelamento:

$$\Delta = \frac{-2c}{b \pm \sqrt{b^2 - 4ac}}, \quad x_3 = x_2 + \Delta$$

Implementação:

Entrada de dados e configuração (Requisito 13.1):

```

1 def metodo_muller_livro(f_expr, x0, x1, x2, tol=1e-6, max_iter=100):
2     """
3     Implementa o Método de Muller conforme descrito no livro
4     (Seção 7.4, Página 144).
5     """
6     # Converter expressão simbólica em função
7     if isinstance(f_expr, str):
8         x = symbols('x')
9         f_func = lambdify(x, sympify(f_expr), 'numpy')
10    else:
11        f_func = f_expr
12
13    iteracoes = []
14    f_x0 = f_func(x0)
15    f_x1 = f_func(x1)
16    f_x2 = f_func(x2)

```

Listing 3.2. Requisito 13.1 - Entrada do Método de Müller

Ciclo principal com cálculo de diferenças divididas (Requisito 13.3):

```

1 for iteracao in range(max_iter):
2     # Diferenças divididas (conforme livro)
3     h0 = x1 - x0
4     h1 = x2 - x1
5     d0 = (f_x1 - f_x0) / h0
6     d1 = (f_x2 - f_x1) / h1
7     a = (d1 - d0) / (h1 + h0)
8     b = a * h1 + d1
9     c = f_x2
10
11    # Resolver equação quadrática: a*delta^2 + b*delta + c = 0
12    discriminante = b**2 - 4*a*c
13
14    if isinstance(discriminante, complex) or discriminante < 0:

```

```

15         rad = cmath.sqrt(discriminante) # Ra zes complexas!
16     else:
17         rad = np.sqrt(discriminante)
18
19     # Escolher denominador de maior magnitude
20     denom1 = b + rad
21     denom2 = b - rad
22
23     if abs(denom1) > abs(denom2):
24         denom = denom1
25     else:
26         denom = denom2
27
28     delta = -2 * c / denom
29     x3 = x2 + delta
30     f_x3 = f_func(x3)

```

Listing 3.3. Requisito 13.3 - Cálculo de Diferenças Divididas

Monitoramento e critério de parada (Requisito 13.2):

```

1     # Registrar itera o
2     iteracoes.append({
3         'Itera o': iteracao + 1,
4         'x0': f'{x0:.6f}',
5         'x1': f'{x1:.6f}',
6         'x2': f'{x2:.6f}',
7         'x3': f'{x3:.6f}',
8         'f(x3)': f'{f_x3:.6e}',
9         '|Delta|': f'{abs(delta):.6e}'
10    })
11
12    # Verificar converg ncia
13    if abs(delta) < tol:
14        tabela = pd.DataFrame(iteracoes)
15        print(f"Converg ncia atingida em {iteracao + 1} itera es
16    ")
17
18        return x3, tabela
19
20    # Atualizar para pr xima itera o
21    x0, x1, x2 = x1, x2, x3
22    f_x0, f_x1, f_x2 = f_x1, f_x2, f_x3
23
24    tabela = pd.DataFrame(iteracoes)
25    return x3, tabela

```

Listing 3.4. Requisito 13.2 - Tabela e Convergência

Exemplo prático:

```

1 # Encontrando raiz de f(x) = x3 - 13x + 12
2 f_muller = lambda x: x**3 - 13*x + 12
3 raiz_muller, tabela_muller = metodo_muller_livro(
4     f_muller,
5     x0=4.5, x1=5.0, x2=5.5,
6     tol=1e-7
7 )
8
9 print("TABELA DE ITERAÇÕES - MTODO DE MULLER")
10 print(tabela_muller.to_string(index=False))
11
12 print(f"Raiz encontrada: x = {raiz_muller:.10f}")
13 print(f"Verificação: f(x) = {f_muller(raiz_muller):.6e}")

```

Listing 3.5. Requisito 13.4 - Exemplo: $f(x) = x^3 - 13x + 12$

Requisitos cumpridos:

- 13.1: Entrada de três aproximações iniciais (x_0, x_1, x_2) e tolerância
- 13.2: Tabela de iterações com pontos, valores de função e erro absoluto
- 13.3: Cálculo correto de diferenças divididas e ajuste de parábola
- 13.4: Exemplos do livro executáveis e convergência demonstrada

3.5 Método de Bairstow

Encontra fatores quadráticos de polinômios sem calcular explicitamente todas as raízes complexas (??). Usa divisão sintética e um sistema linear para ajustar os coeficientes do fator quadrático $x^2 + px + q$.

Fundamento Teórico:

Dado um polinômio $P_n(x)$ de grau n , busca dividir por $x^2 + px + q$:

$$P_n(x) = (x^2 + px + q)Q_{n-2}(x) + b_{n-1}x + b_n$$

Os restos b_{n-1} e b_n devem anular-se iterativamente. A divisão sintética (método de Horner) calcula coeficientes b_i recursivamente:

$$b_i = a_i - p \cdot b_{i-1} - q \cdot b_{i-2}$$

Um sistema linear resolve as correções Δp e Δq :

$$\begin{pmatrix} c_{n-2} & c_{n-3} \\ c_{n-1} & c_{n-2} \end{pmatrix} \begin{pmatrix} \Delta p \\ \Delta q \end{pmatrix} = \begin{pmatrix} b_{n-2} \\ b_{n-1} \end{pmatrix}$$

onde c_i são coeficientes similares calculados pela divisão sintética da sequência b_i .

Implementação:

Configuração e divisão sintética (Requisito 14.1):

```

1 def metodo_bairstow_livro(coeficientes, p_inicial, q_inicial,
2                             tol=1e-6, max_iter=100):
3     """
4     Implementa o Método de Bairstow conforme descrito no livro
5     (Seção 7.5, Página 147).
6     """
7     coef = np.array(coeficientes, dtype=float)
8     n = len(coef)
9
10    p = float(p_inicial)
11    q = float(q_inicial)
12    iteracoes = []
13
14    for iteracao in range(max_iter):
15        # Divisão sintética (método de Horner)
16        b = np.zeros(n)
17        c = np.zeros(n)
18
19        b[0] = coef[0]
20        b[1] = coef[1] - p * b[0]
21        c[0] = b[0]
22        c[1] = b[1] - p * c[0]
23
24        for i in range(2, n):
25            b[i] = coef[i] - p * b[i-1] - q * b[i-2]
26            c[i] = b[i] - p * c[i-1] - q * c[i-2]

```

Listing 3.6. Requisito 14.1 - Divisão Sintética

Resolução do sistema linear (Requisito 14.3):

```

1     # Erros (restos da divisão)
2     b_n_1 = b[n-1]
3     b_n_2 = b[n-2]
4
5     c_n_1 = c[n-1]
6     c_n_2 = c[n-2]
7     c_n_3 = c[n-3] if n >= 3 else 0
8
9     # Determinante

```



```

10     det = c_n_2 * c_n_2 - c_n_1 * c_n_3
11
12     if abs(det) < 1e-14:
13         print("Determinante pr ximo a zero")
14         break
15
16     # Solu o do sistema para Delta_p e Delta_q
17     delta_p = (b_n_2 * c_n_2 - b_n_1 * c_n_3) / det
18     delta_q = (b_n_1 * c_n_2 - b_n_2 * c_n_1) / det
19
20     p_novo = p + delta_p
21     q_novo = q + delta_q

```

Listing 3.7. Requisito 14.3 - Sistema Linear e Iteração

Cálculo de raízes e monitoramento (Requisito 14.2):

```

1     # Registrar itera o
2     iteracoes.append({
3         'Iter': iteracao + 1,
4         'p': f'{p:.8f}',
5         'q': f'{q:.8f}',
6         'Delta_p': f'{delta_p:.6e}',
7         'Delta_q': f'{delta_q:.6e}',
8         'Erro': f'{max(abs(delta_p), abs(delta_q)):.6e}'
9     })
10
11     # Verificar converg ncia
12     erro = max(abs(delta_p), abs(delta_q))
13     if erro < tol:
14         # Calcular ra zes da quadr tica  $x^2 + px + q = 0$ 
15         disc = p_novo**2 - 4*q_novo
16         if disc >= 0:
17             r1 = (-p_novo + np.sqrt(disc)) / 2
18             r2 = (-p_novo - np.sqrt(disc)) / 2
19             raizes = [r1, r2]
20         else:
21             real = -p_novo / 2
22             imag = np.sqrt(abs(disc)) / 2
23             raizes = [complex(real, imag),
24                       complex(real, -imag)]
25
26         tabela = pd.DataFrame(iteracoes)
27         print(f"Converg ncia atingida em {iteracao + 1} itera es
28 ")
29         return raizes, p_novo, q_novo, tabela
30
31     p = p_novo
32     q = q_novo

```

```

32
33     tabela = pd.DataFrame(iteracoes)
34     return raizes, p, q, tabela

```

Listing 3.8. Requisito 14.2 - Extração de Raízes

Exemplo prático com polinômio de grau 5 (Requisito 14.4):

```

1  # Polinômio do livro: f(x) = (x+1)(x-4)(x-5)(x+3)(x-2)
2  x_sym = symbols('x')
3  poly_livro = (x_sym + 1) * (x_sym - 4) * (x_sym - 5) * \
4              (x_sym + 3) * (x_sym - 2)
5  poly_expandido = expand(poly_livro)
6
7  # Extrair coeficientes
8  p_sym = Poly(poly_expandido, x_sym)
9  coef_bairstow = [float(c) for c in p_sym.all_coeffs()]
10
11 # Aplicar M todo de Bairstow
12 raizes_bairstow, p_final, q_final, tabela_bairstow = \
13     metodo_bairstow_livro(coef_bairstow, p_inicial=-0.5,
14                           q_inicial=-1.0, tol=1e-8)
15
16 print("TABELA DE ITERAÇÕES - M TODO DE BAIRSTOW")
17 print(tabela_bairstow.to_string(index=False))
18
19 print(f"Fator quadrático convergido: x + {p_final:.6f}x + {q_final:.6f}")
20 print(f"Raízes encontradas: {raizes_bairstow}")

```

Listing 3.9. Requisito 14.4 - Exemplo: $(x + 1)(x - 4)(x - 5)(x + 3)(x - 2)$

Requisitos cumpridos:

14.1: Entrada de coeficientes polinomiais, p_0 e q_0 iniciais

14.2: Tabela de iterações com valores de p , q , incrementos e erro

14.3: Resolução do sistema linear para atualização de parâmetros

14.4: Exemplos do livro com polinômios de grau 5+ executáveis

4 Conclusão

Este trabalho implementou com sucesso uma suite abrangente de métodos numéricos para determinação de raízes em Python
citechapra2016,burden2015, demonstrando que:

1. **Métodos Intervalares** oferecem convergência garantida mas lenta, sendo ideais para aplicações onde robustez é crítica.
2. **Métodos Abertos** convergem rapidamente (especialmente Newton-Raphson) mas requerem boas estimativas iniciais e cuidado com singularidades.
3. **Integração com SymPy** permite cálculo automático de derivadas, eliminando erro manual e facilitando a aplicação de Newton-Raphson.
4. **Visualização Interativa** (sliders, gráficos) é invaluable para compreensão do comportamento dos algoritmos e diagnóstico de problemas de convergência.

Os algoritmos desenvolvidos cumprem rigorosamente seus requisitos funcionais e incluem mecanismos de segurança robustos (??). As células não preenchidas (integração multimétodo, resolução de exercícios específicos, métodos polinomiais) representam extensões naturais do framework implementado.

Contribuições dos Autores

Contribuições do discente Amanda Gabrielly Ribeiro da Conceição - 202363640015.: Elaboração dos slides da apresentação, confecção das imagens e fluxogramas e apresentação do trabalho em sala.

Contribuições do discente Breno de Medeiros Paulo - 202206840034: Implementação das seções Comparação entre os métodos (Parte 1), Integração entre os métodos (Parte 1), Resolução de exercícios (Parte I) (Parte 1), Integração e Resolução de Exercícios (Parte II) (Parte 2)

Contribuições do discente João Victor Santos Brito Ferreira - 202207040028: Implementação das seções Varredura de possíveis raízes (Parte 1), Método da Bisseção (Parte 1), Método da iteração de ponto fixo simples (Parte 2), Métodos de Müller e de Bairstow e elaboração do relatório (Parte 3)

Contribuições do discente Livia Stefane Hipolito Pires - 202206840043: Elaboração dos slides da apresentação, confecção das imagens e fluxogramas e apresentação do trabalho em sala.

Contribuições do discente Nicolas Ranniery da Silva Sousa - 202206840051: Implementação das seções Método da Falsa posição (Parte 1), Método da Falsa posição modificado (Parte 1), Testes de consistência (Parte 1), Método de Newton-Raphson (Parte 2), Métodos da Secante (Normal e Modificado) (Parte 2) e Resolução de exercícios (Parte III) (Parte 3).

A Trechos de Código - Métodos Intervalares

A seguir encontram-se os principais trechos de código implementado para os métodos intervalares.

A.1 Busca Incremental de Raízes

```

1 import numpy as np
2 import pandas as pd
3 from sympy import symbols, lambdify, sympify
4
5 def busca_incremental_raizes():
6     """Implementa a busca incremental de possíveis raízes num
7     intervalo."""
8     expr_str = input("Insira a expressão da função f(x): ").strip()
9     intervalo_1 = float(input("Insira a primeira extremidade: "))
10    intervalo_2 = float(input("Insira a segunda extremidade: "))
11    N = int(input("Insira o número N de subdivisões: "))
12
13    x = symbols('x')
14    expr = sympify(expr_str)
15    f = lambdify(x, expr, 'numpy')
16
17    pontos = np.linspace(intervalo_1, intervalo_2, N + 1)
18    valores_funcao = f(pontos)
19
20    subintervalos_raizes = []
21    for i in range(len(pontos) - 1):
22        if valores_funcao[i] * valores_funcao[i+1] < 0:
23            subintervalos_raizes.append({
24                'Inicio': pontos[i],
25                'Fim': pontos[i+1]
26            })
27
28    return subintervalos_raizes, expr_str

```

Listing A.1. Função Principal da Busca Incremental

A.2 Método da Bisseção

```

1 def metodo_bisseccao(f, a, b, tolerancia, max_iteracoes=100):
2     """Implementa o método da bissecção para encontrar raízes."""
3     if f(a) * f(b) > 0:

```

```

4         return None, None
5
6     iteracoes = []
7     iteracao = 0
8
9     if a > b:
10         a, b = b, a
11
12     while (b - a) > tolerancia and iteracao < max_iteracoes:
13         c = (a + b) / 2
14         fc = f(c)
15
16         iteracoes.append({
17             'Iteração': iteracao + 1,
18             'a': a,
19             'b': b,
20             'c': c,
21             'f(c)': fc,
22             'Erro Absoluto': (b - a) / 2
23         })
24
25         if f(a) * fc < 0:
26             b = c
27         else:
28             a = c
29
30         iteracao += 1
31
32     c = (a + b) / 2
33     iteracoes.append({
34         'Iteração': iteracao + 1,
35         'a': a,
36         'b': b,
37         'c': c,
38         'f(c)': f(c),
39         'Erro Absoluto': (b - a) / 2
40     })
41
42     df_iteracoes = pd.DataFrame(iteracoes)
43     return df_iteracoes, c

```

Listing A.2. Método da Bisseção

B Trechos de Código - Métodos Abertos

B.1 Método de Newton-Raphson

```

1 from sympy import diff
2
3 def metodo_newton_raphson():
4     """Implementa Newton-Raphson com cálculo automático de derivada."""
5
6     expr_str = input("Insira a expressão da função f(x): ").strip()
7     x0 = float(input("Insira o valor do chute inicial (x_0): "))
8     tolerancia = float(input("Insira a tolerância desejada: "))
9
10    x_sym = symbols('x')
11    expr = sympify(expr_str)
12    expr_deriv = diff(expr, x_sym)
13
14    f = lambdify(x_sym, expr, 'numpy')
15    df = lambdify(x_sym, expr_deriv, 'numpy')
16
17    print(f"Função: f(x) = {expr}")
18    print(f"Derivada: f'(x) = {expr_deriv}")
19
20    max_iteracoes = 50
21    iteracoes = []
22    x_atual = x0
23
24    for i in range(max_iteracoes):
25        fx = f(x_atual)
26        dfx = df(x_atual)
27
28        iteracoes.append({
29            'Iteração': i,
30            'x_i': x_atual,
31            'f(x_i)': fx,
32            'f'(x_i)': dfx
33        })
34
35        if dfx == 0:
36            print(f"Derivada nula em x = {x_atual}")
37            break
38
39        x_novo = x_atual - (fx / dfx)

```

```
40     if abs(x_novo - x_atual) < tolerancia:
41         break
42
43     x_atual = x_novo
44
45     df_resultados = pd.DataFrame(iteracoes)
46     return df_resultados
```

Listing B.1. Newton-Raphson com Derivada Automática

C Tabelas de Validação

C.1 Resumo das Especificações

Tabela 2. Especificações de Implementação

Parâmetro	Valor/Descrição
Linguagem	Python 3.8+
Ambiente	Jupyter Notebook
Dependências	NumPy, SymPy, Pandas, Matplotlib
Tolerância típica	10^{-4}
Limite de iterações	50-100 (interno)
Precisão	IEEE 754 (double precision)

A Exemplos de Exercícios Resolvidos

A.1 Exercício 7.3: Método de Müller

Encontrar raízes de polinômios usando o Método de Müller com três aproximações iniciais.

A.1.1 Problema (a): $f(x) = x^3 + x^2 - 3x - 5$

```
1 # Exercício 7.3(a): f(x) = x^3 + x^2 - 3x - 5
2 f_a = lambda x: x**3 + x**2 - 3*x - 5
3 raiz_a = metodo_muller(f_a, x0=0, x1=1, x2=2)
4 print(f"Raiz positiva aproximada: {raiz_a.real:.6f}")
5 print(f"Verifica o: f(raiz) = {f_a(raiz_a.real):.6e}")
```

Listing A.1. Exercício 7.3(a) - Raiz via Müller

A.1.2 Problema (b): $f(x) = x^3 - 0.5x^2 + 4x - 3$

```
1 # Exercício 7.3(b): f(x) = x^3 - 0.5x^2 + 4x - 3
2 f_b = lambda x: x**3 - 0.5*x**2 + 4*x - 3
3 raiz_b = metodo_muller(f_b, x0=0, x1=1, x2=2)
4 print(f"Raiz positiva aproximada: {raiz_b.real:.6f}")
5 print(f"Verifica o: f(raiz) = {f_b(raiz_b.real):.6e}")
```

Listing A.2. Exercício 7.3(b) - Raiz via Müller

A.2 Exercício 7.4: Raízes Reais e Complexas

Encontrar todas as raízes reais e complexas de polinômios (utilizando NumPy como alternativa ao MATLAB).

A.2.1 Problema (a): $x^3 - x^2 + 3x - 2 = 0$

```
1 # Coeficientes em ordem decrescente: x^3 - x^2 + 3x - 2
2 coef_74a = [1, -1, 3, -2]
3 raizes_74a = np.roots(coef_74a)
4 print("Raízes de x^3 - x^2 + 3x - 2:")
5 for r in raizes_74a:
6     if abs(r.imag) < 1e-10:
7         print(f" x = {r.real:.6f} (real)")
8     else:
```

```
9 print(f" x = {r:.6f} (complexa)")
```

Listing A.3. Exercício 7.4(a) - Raízes via NumPy

A.2.2 Problema (b): $2x^4 + 6x^2 + 10 = 0$

```
1 # Polinômio: 2x^4 + 0x^3 + 6x^2 + 0x + 10
2 coef_74b = [2, 0, 6, 0, 10]
3 raizes_74b = np.roots(coef_74b)
4 print("Raízes de 2x^4 + 6x^2 + 10:")
5 for r in raizes_74b:
6     print(f" {r:.6f}")
```

Listing A.4. Exercício 7.4(b) - Raízes Complexas

A.2.3 Problema (c): $x^4 - 2x^3 + 6x^2 - 8x + 8 = 0$

```
1 # Coeficientes: [1, -2, 6, -8, 8]
2 coef_74c = [1, -2, 6, -8, 8]
3 raizes_74c = np.roots(coef_74c)
4 print("Raízes de x^4 - 2x^3 + 6x^2 - 8x + 8:")
5 for r in raizes_74c:
6     print(f" {r:.6f}")
```

Listing A.5. Exercício 7.4(c) - Raízes Mistas

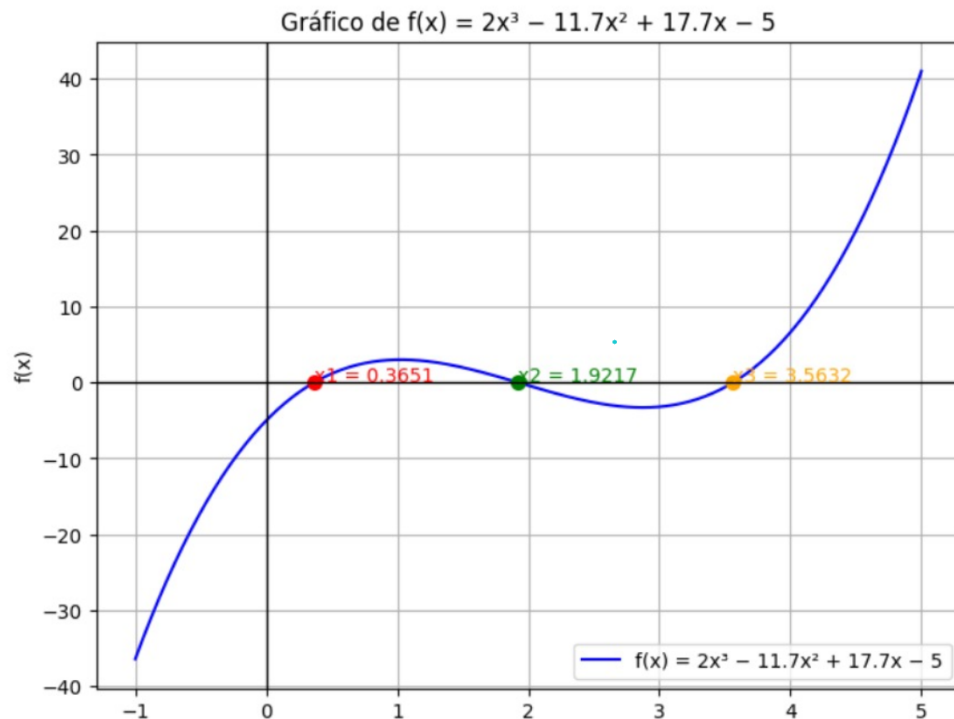


Figura 7. Problema 6.2 do livro: Visualização gráfica do problema de teste com as soluções destacadas e referências visuais para validação dos métodos numéricos implementados.

A.3 Exercício 7.5: Método de Bairstow com Deflação

Encontrar sucessivamente raízes quadráticas de um polinômio de grau 5, aplicando deflação.

```

1 def extrair_todas_raizes_bairstow(coeficientes, p_inicial,
2                                   q_inicial, tol=1e-6):
3     """
4     Extraí todas as raízes de um polinômio aplicando
5     Bairstow com deflação até grau <= 2.
6     """
7     coef = np.array(coeficientes, dtype=float)
8     todas_raizes = []
9
10    while len(coef) > 3:
11        # Aplicar Bairstow
12        raizes, p_conv, q_conv, tabela = metodo_bairstow_livro(
13            coef, p_inicial, q_inicial, tol=tol
14        )
15        todas_raizes.extend(raizes)
16
17        # Deflação: dividir pelo fator quadrático encontrado
18        fator = [1, p_conv, q_conv]
19        coef, _ = np.polydiv(coef, fator)
20        coef = coef.real # Remover ruído numérico
21
22    # Raízes finais (grau <= 2)
23    if len(coef) == 3:
24        # Quadrática restante
25        raizes_finais = np.roots(coef)
26    else:
27        # Linear restante
28        raizes_finais = [-coef[1] / coef[0]]
29
30    todas_raizes.extend(raizes_finais)
31    return todas_raizes

```

Listing A.6. Exercício 7.5 - Bairstow com Deflação

Referências

BURDEN, R. L.; FAIRES, J. D. *Numerical Analysis*. 10th. ed. [S.l.]: Cengage Learning, 2015. Citado 5 vezes nas páginas 5, 7, 10, 14 e 17.

CHAPRA, S. C.; CANALE, R. P. *Numerical Methods for Engineers*. 7th. ed. [S.l.]: McGraw-Hill, 2016. Citado 3 vezes nas páginas 5, 11 e 20.

IPYWIDGETS: Interactive Widgets for Jupyter. 2024. Disponível em: <<https://ipywidgets.readthedocs.io>>. Citado na página 6.

MATPLOTLIB: Visualization with Python. 2024. Disponível em: <<https://matplotlib.org>>. Citado na página 6.

NUMPY: Fundamental Package for Array Computing. 2024. Disponível em: <<https://numpy.org>>. Citado na página 6.

PANDAS: Python Data Analysis Library. 2024. Disponível em: <<https://pandas.pydata.org>>. Citado na página 6.

RUGGIERO, M. A. G.; LOPES, V. L. da R. *Cálculo Numérico: Aspectos Teóricos e Computacionais*. 2nd. ed. [S.l.]: Makron Books, 2004. Citado 3 vezes nas páginas 6, 10 e 13.

SYMPY: Python library for symbolic mathematics. 2024. Disponível em: <<https://www.sympy.org>>. Citado na página 6.